esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

Services Web REST avec Java JAX RS

Module SOA A.U 2023-2024

United Nations Educational, Scientific and Cultural Organization
UNESCO Chair "Project-based learning" ESPRIT School of engineering, Tunisia
uniTwin
EUR-ACE® Délivrée par la Commission des Titres d'Ingénieur
CONFERENCE DES GRANDES ÉCOLES
cdio CONCEIVE DESIGN IMPLEMENT OPERATE

Services Web RESTavec Java

- JAX-RS: **J**ava **A**PI for **R**ESTful Web **S**ervices
- Spécification décrivant la mise en œuvre et la consommation des services web REST
- JAX-RS est basé sur des annotations Java
- Pas de possibilité de développer le service à partir d'un WADL (contrairement à SOAP):
  - Seule **l'approche bottom-up** est possible:
    - Annoter une classe POJO
    - Compiler et déployer

Services Web REST avec Java

- Différentes implémentations de JAX-RS sont disponibles:
  - JERSEY (Oracle)
  - CXF (Apache)
  - RESTEasy (JBoss)
  - RESTlet

Annotations JAX-RS

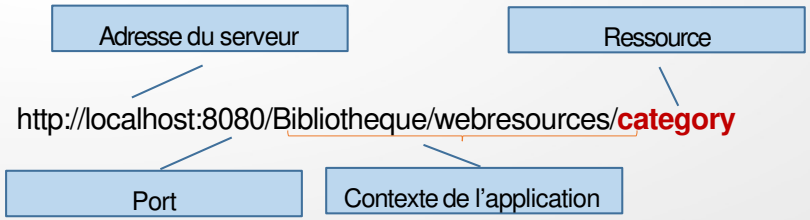
- La spécification JAX-RS dispose d'un ensemble d'annotation permettant **d'exposer** *une classe java sous forme d'un service web* :

@Path	Définit le chemin de la ressource. Cette annotation se place sur la classe et/ou sur la méthode implémentant le service.
@GET, @PUT, @POST, @DELETE	Définit l'action implémentée par le service
@Produces	Spécifie le type de la réponse du service
@Consumes	Spécifie le type accepté en entré du service

Annotation @PATH

- L'annotation **@PATH** permet de rendre une classe accessible par une requête HTTP
- Elle définit la racine des ressources (Root Racine Ressources)
- La valeur donnée correspond à l'URI relative de la ressource

```
@Path("category")
public class CategoryFacade {
……
}
```



Annotation @PATH

- L'annotation peut être utilisée pour annoter des méthodes d'une classe
- L'URI résultante est la concaténation entre le valeur de @path de la classe et celle de la méthode:

```
@Path("category")
public class CategoryFacade {
  @GET
  @Produces(MediaType.APPLICATION_XML)

  @Path("helloTest")
  public String hello(){
     return "Hello World!";
  }
..
}
```

http://localhost:8080/Bibliotheque/webresources/**category**/**helloTest**

Annotation @GET, @POST, @PUT, @DELETE

- Permettent de mapper une méthode à un type de requête HTTP
- Ne sont utilisables que sur des méthodes
- Le nom de la méthode n'a pas d'importance, JAX-RS détermine la méthode à exécuter en fonction de la requête

```
@Path("category")
public class CategoryFacade {
@GET
@Produces({MediaType.APPLICATION_XM
L)  @Path("test")
public String hello(){
   return "Hello World!";
}
..
}
```

```
@GET
@Produces                    LICATION_JSON)
(MediaType.APP
@Path("hello/{nom}")         thParam("nom") String nom){
public String hello
(@Pa
   return "Hello " + nom;
}
```

Annotation @GET, @POST, @PUT, @DELETE

- Les opérations CRUD sur les ressources sont réalisées au travers des méthodes de la requête HTTP



GET, POST
PUT, DELETE

/books
GET: Liste des livres
POST: Créer un nouveau livre

/books/{id}
GET: Livre identifié par l'id
PUT: Mis à jour du livre identifié par id
DELETE: Supprimer le livre identifié par id

Paramètres des requêtes

- JAX-RS fournit des mécanismes pour extraire des paramètres dans la requête
- Utilisés sur les paramètres des méthodes des ressources pour réaliser des injections de contenu
- Différentes annotations :
  - @PathParam : valeurs dans templates parameters
  - @QueryParam : valeurs des paramètres de la requête
  - @FormParam : Valeurs des paramètres de formulaire
  - @HeaderParam: Valeurs dans l'en tète de la requête
  - @DefaultValue: valeur définie par défaut pour les annotations ci-dessus lorsque la clé est introuvable.
  - @CookieParam : Valeurs des cookies

Annotation @PathParam

- La valeur définie dans l'annotation @Path n'est forcément une constante, elle peut être variable.
- Possibilité de définir des expressions plus complexes, appelées **Template Parameters** qui seront récupérés par la suite coté serveur.
- Les contenus complexes sont délimités par « **{}** »
- Possibilité de mixer dans la valeur **@Path** des expressions

```
@GET
@Produces ({MediaType.APPLICATION_JSON,
MediaType.APPLICATION_XML})

@Path("hello/{nom}")
public String hello (@PathParam("nom") String nom){
  return "Hello " + nom;
}
```

http://localhost:8080/Bibliotheque/webresources/category/**hello**/**Miage**

Annotation @QueryParam

- L'annotation @QueryParam permet de récupérer des paramètres passés à l'URI côté serveur

```
@GET
@Produces ( MediaType.APPLICATION_XML)

@Path("hello")

public String hello (@QueryParam("from") String exp, @QueryParam("to") String dest){
   return "Hello to" + dest + "from me: " + exp;
}
```

- Accéder à l'URL suivant:

  http://localhost:8080/Bibliotheque/webresources/category/**hello?from=Sarra&to=Celine**

Mise en place du projet

- Pour indiquer le chemin de votre application, vous devez commencer par la création d'une sous-classe **javax.ws.rs.core.Application** (classe qui étend la classe Application).
- L'annotation @ApplicationPath va mapper toutes les ressources de votre application si elles existent
- Créer une classe de test, dans notre exemple la classe **CategoryFacade**

```
import javax.ws.rs.core.Application;
import javax.ws.rs.ApplicationPath;

@ApplicationPath("Webressource")
public class CategApplication extends Application {

}
```

```
@Path("category")
public class CategoryFacade {
 @GET
 @Produces(.....)
 @Path("test")
 public String hello(){
    return "Hello World!";
 }
}
```

- Lancer l'application avec Tomcat et accéder à l'URL suivant :

http://localhost:8080/Bibliotheque/**Webressource**/category/test